Gesture detection with

Infineon`s CY8CKIT-062S2-AI AI Evaluation Kit

Application note

Versions

Version	Date	Rationale
1.0	April 13, 2026	First release. Author: ROJ, KOA

Legal disclaimer

The evaluation board is for testing purposes only and, because it has limited functions and limited resilience, is not suitable for permanent use under real conditions. If the evaluation board is nevertheless used under real conditions, this is done at one's responsibility;
any liability of Rutronik is insofar excluded.

Table of contents

Table of contents 2

Introduction 3

Useful links 3

Getting started 3

Algorithm 4

Azimuth and elevation signals 5

Changing the configuration 5

Adding new motions 6

Implementation in firmware 7

Introduction

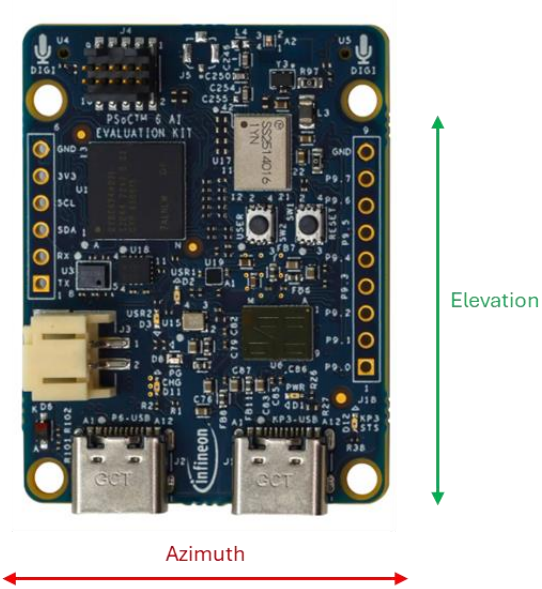
This document describes the calculations behind the detection of position of the user´s hand when they “click” in front of the radar sensor. The 062S2-AI evaluation kit from Infineon is used for it. In the related project, the GUI collects the raw sensor data, apply for signal processing on it and displays it on the screen.

Useful links

1. GitHub project for gesture detection with step-by-step guide of the operation: https://github.com/RutronikSystemSolutions/CY8CKIT_062S2_AI_Radar_Gesture
2. Application note to understand the basics of digital signal processing for FMCW radar (range FFT and doppler FFT): https://github.com/RutronikSystemSolutions/RAB3-Radar_Documents/blob/main/RAB3-Radar_AppNote_Signal_Processing.pdf
3. Examples of the source code (range FFT, Doppler FFT, phase difference, …): https://github.com/RutronikSystemSolutions/trainings/tree/main/radar_dsp/src/c

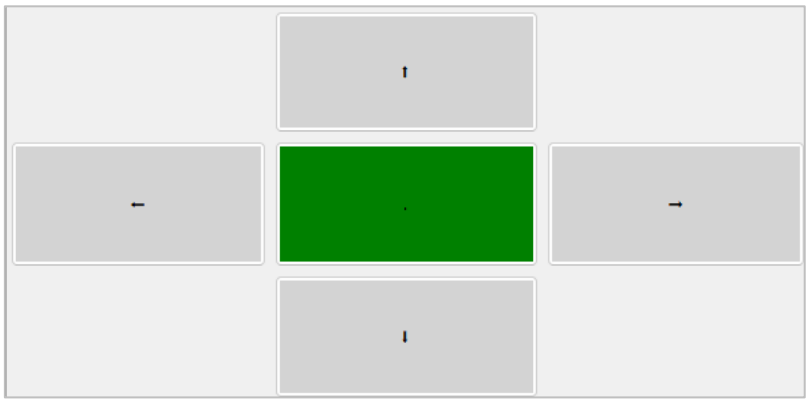
Getting started

To detect the gestures correctly, use the same orientation for the board as shown below (USB ports facing down).

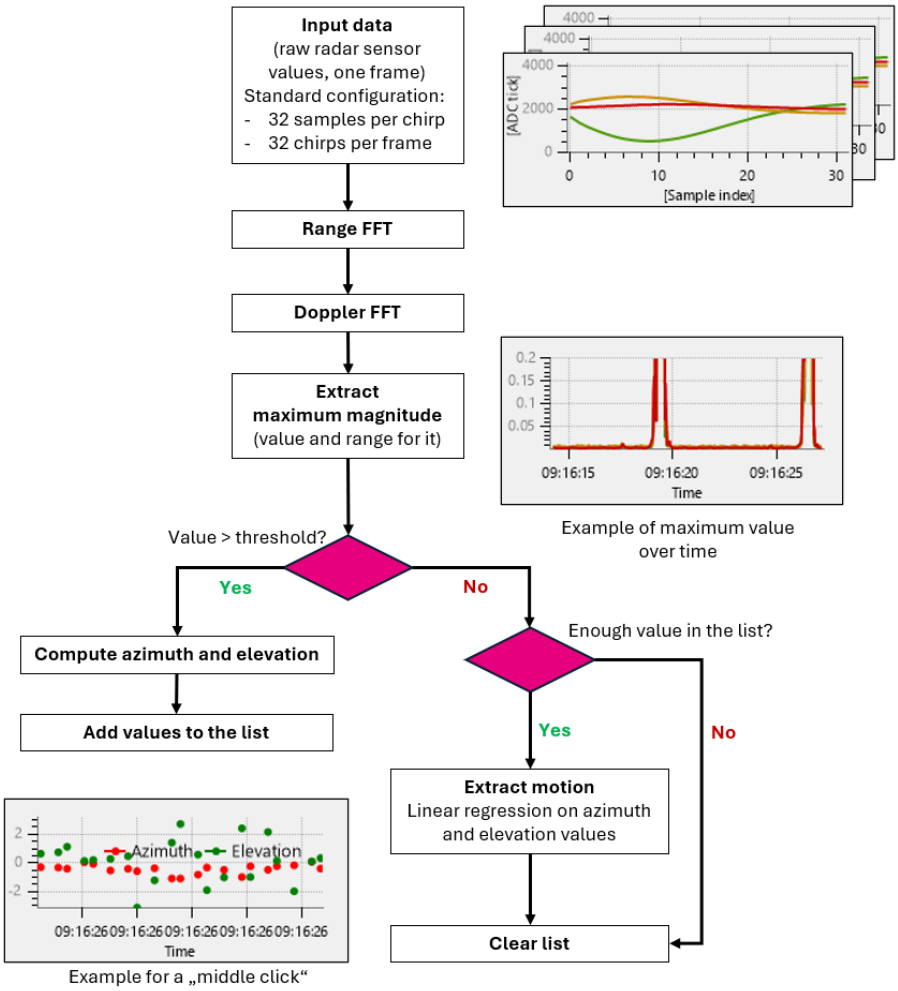


The measuring the angle of arrival only works when the object is moving as it uses the result of Doppler FFT for computation. Small motion is enough for that.

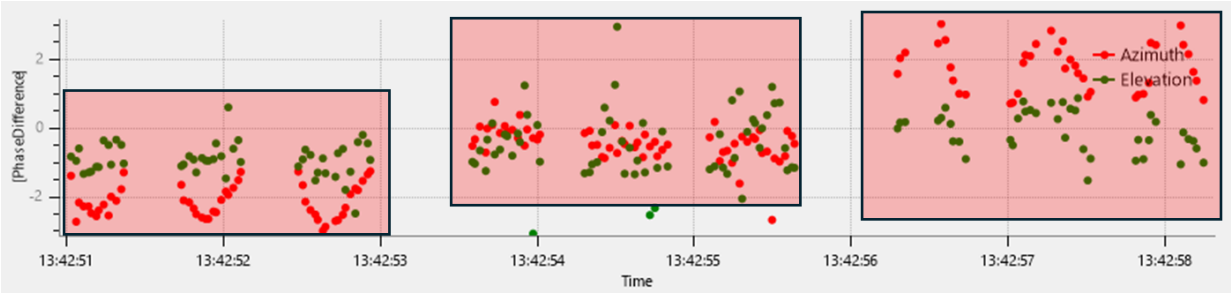
The output in the GUI looks like the picture below (green is the detected gesture).



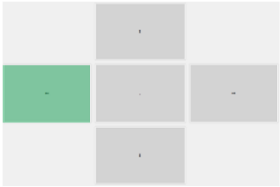
Algorithm



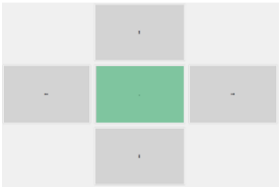
Azimuth and elevation signals



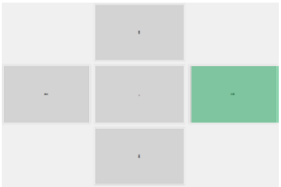
3x click on the "left"



3x click in the "middle"



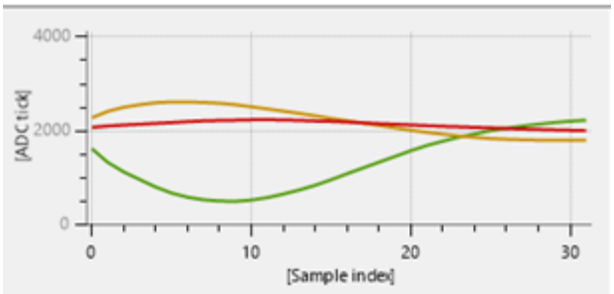
3x click on the "right"



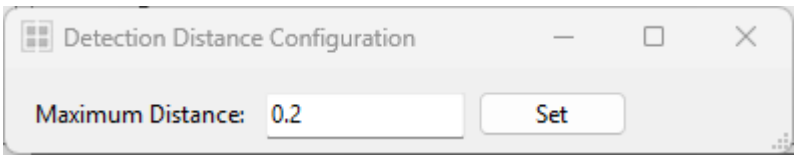
Changing the configuration

The current system is made for "near" detection (20cm). You can adapt the system to work with larger distance by changing the radar configuration.

You can do this by modifying the "radar_settings.h" file in the project (Radar Fusion GUI needed for that).



Make sure that the raw sensor values are not "saturated" (below 0 or more than 4095). Then you need to change the software configuration of the gesture detector:



By default, it is set to 0.2m (20cm).

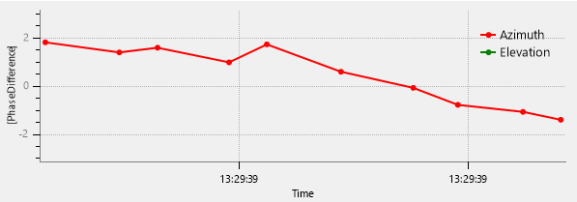
Adding new motions

Now, the algorithm only detects clicks (and will “reject” swipe movements). This is done in the file “GestureClassificator.cs”:

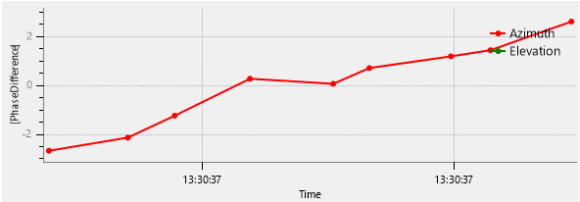
```
LinearRegression.Compute(azimuths, out azimuthSlope, out azimuthIntercept);
LinearRegression.Compute(elevations, out elevationSlope, out elevationIntercept);

if (Math.Abs(azimuthSlope) < 0.2 && Math.Abs(elevationSlope) < 0.2)
```

A linear regression is computed, and if the slope is small enough (i.e. no right/left – bottom/up movement) then we search for the matching click.

Of course, it is possible to detect “swipes” motion. See an example of “azimuth” signal below, with left/right and right/left motions:

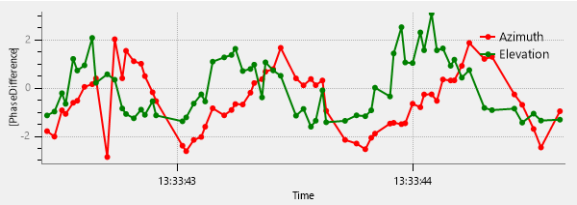


A “right to left” motion

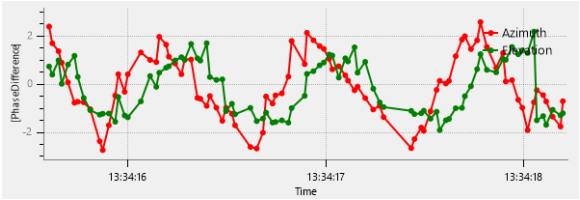


A “left to right” motion

You can also implement the “circle” motions:



Clockwise motion



Anti-clockwise motion

Implementation in firmware

For a better flexibility, the algorithm is implemented in C# now, but you can choose to implement it on the PSOC micro-controller directly (the needed DSP libraries are already integrated inside the project).

Example below to compute the range FFT of the first chirp, for the antenna RX1:

```
// Do extraction of the first chirp of the first antenna
for (int i = 0; i < XENSIV_BGT60TRXX_CONF_NUM_SAMPLES_PER_CHIRP; ++i)
{
    // buffer_raw[0] -> sample 0 of chirp 0 of RX 0 -> first_chirp[0]
    // buffer_raw[1] -> sample 0 of chirp 0 of RX 1
    // buffer_raw[2] -> sample 0 of chirp 0 of RX 2
    // buffer_raw[3] -> sample 1 of chirp 0 of RX 0 -> first_chirp[1]
    // ...
    int start_index = (i * XENSIV_BGT60TRXX_CONF_NUM_RX_ANTENNAS);
    first_chirp[i] = radar_raw_values[start_index];

    // Scaling -> (0 -> 1)
    first_chirp[i] = first_chirp[i] / 4095.f;
} // for i

// Compute Range FFT
arm_rfft_fast_f32(&rfft, first_chirp, (float32_t*)range, 0);
CIMAG_F32(range[0]) = 0.0f;
```

You can find more examples of the source code on our GitHub, at this address:

https://github.com/RutronikSystemSolutions/trainings/tree/main/radar_dsp/src/c